

Service Mesh + Istio + Spring Boot on AWS EKS — Interview Conversational Script

"Flipkart Microservices" as the Production Example

Written for: New Learner → Architect Level

HOW TO USE THIS FILE

This file builds from zero. If you are new, read SECTION 1 first — it explains everything in plain English with before/after comparisons. Then read the deeper sections which go into how each feature is actually implemented in production.

Each section shows:

- The PROBLEM (without service mesh)
 - The SOLUTION (with service mesh)
 - How to explain it in an interview
 - ASCII diagram of what actually happens
-

SECTION 1 — NEW LEARNER FOUNDATION

Understanding the Problem Before Learning the Solution

STORY: Flipkart Without Service Mesh

Start here if you are new. Read this like a story.

Imagine you just joined Flipkart's engineering team. Flipkart has broken their application into microservices. Here is a simplified picture of what they have:

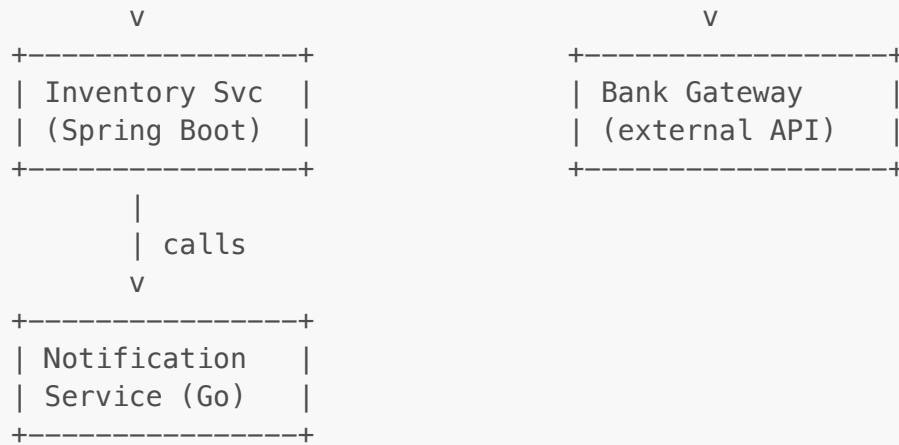
FLIPKART — MICROSERVICES OVERVIEW

=====

User places an order on Flipkart app

|
v





These services talk to each other over HTTP inside the Kubernetes cluster. Now let's look at the five real problems Flipkart faces — and how they solved each one without a service mesh, and then WITH a service mesh.

PROBLEM 1 — Security: Services Talk Over Plain HTTP Inside the Cluster

WITHOUT Service Mesh

All traffic between services is plain HTTP — not encrypted.

This means: if an attacker gets inside the Kubernetes cluster, they can read every request and response between services. When order-service sends payment details to payment-service — card number, amount, user ID — it goes as plain text on the network.

WITHOUT SERVICE MESH – Plain HTTP Inside Cluster

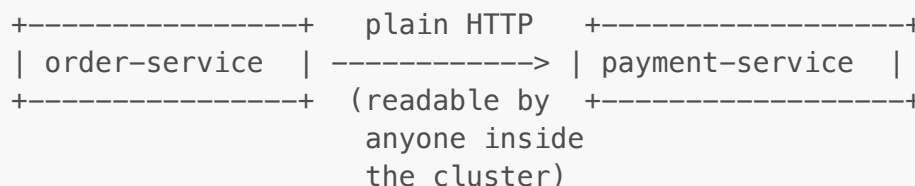
=====

order-service sends:

POST http://payment-service/pay

Body: { "cardNumber": "4111...", "amount": 5000 }

This travels as PLAIN TEXT through the Kubernetes network.



Attacker inside cluster:

- Can read card numbers
- Can modify requests
- Can impersonate services

How teams tried to fix this WITHOUT service mesh:

Each Spring Boot service added TLS configuration manually. They loaded keystores, configured SSL, wrote certificate rotation code. Every service. Every language. Hundreds of services. Painful.

WITH Service Mesh (Istio mTLS)

Istio automatically encrypts ALL traffic between services using mTLS. No code change in any Spring Boot service.

WITH SERVICE MESH – All Traffic Encrypted Automatically

=====

order-service code still calls:

POST http://payment-service/pay <- still plain HTTP in the code

But what actually travels on the network:

```

+-----+      mTLS (encrypted)      +-----+
| order-service | -----> | payment-service |
| + Envoy proxy | TLS 1.3, certs | + Envoy proxy |
+-----+      auto-managed          +-----+
```

Attacker inside cluster:

- Sees only encrypted bytes – useless
- Cannot impersonate services – certificates verify identity

What changed in Spring Boot code: NOTHING

What changed in Kubernetes yaml: NOTHING (except namespace label)

PROBLEM 2 — Reliability: What Happens When Payment Service is Slow or Down?

WITHOUT Service Mesh

When payment-service is slow, order-service waits. And waits. The request times out. The user sees an error. Worse — if payment-service crashes and comes back up, order-service does not retry. The order just fails.

So every developer adds retry logic inside their own service — in Java, in Go, in Python. Every service writes the same retry code differently.

WITHOUT SERVICE MESH – Manual Retry Code in Every Service

=====

```
// OrderService.java – developer has to write this
@Service
public class OrderService {

    public PaymentResult pay(PaymentRequest request) {
        int attempts = 0;
```

```

        while (attempts < 3) {
            try {
                return paymentClient.pay(request); // may fail
            } catch (Exception e) {
                attempts++;
                if (attempts == 3) throw e;
                Thread.sleep(100 * attempts); // backoff
            }
        }
    }
}

// Now write the same for EVERY service that calls EVERY other service
// inventory-service: same retry code
// notification-service (Go): same retry code but in Go
// 50 services x 3 dependencies = 150 places to write and maintain retry
logic

```

WITH Service Mesh (Istio Retries)

Istio handles retries transparently at the proxy level. Order-service calls payment-service once. Envoy retries automatically if it fails. Order-service code has no retry logic at all.

WITH SERVICE MESH – Retry Config in One YAML File

=====

```

# One yaml file, covers ALL callers of payment-service
retries:
  attempts: 3
  perTryTimeout: 2s
  retryOn: "connect-failure,503"

```

Order-service Spring Boot code:

```
return paymentClient.pay(request); // just one line, no retry code
```

What Envoy does automatically:

Attempt 1: payment-service 503 -> wait 100ms

Attempt 2: payment-service 503 -> wait 200ms

Attempt 3: payment-service 200 OK -> return result to order-service

Order-service received: 200 OK

Order-service code knew nothing about the 2 retries

50 services: ZERO retry code written

PROBLEM 3 — Canary Deployment: Testing New Version on Real Traffic

WITHOUT Service Mesh

Flipkart wants to release payment-service v2.0. They want to test it with only 10% of real traffic before full rollout.

Without a service mesh, the only way to do this in Kubernetes is to run 9 pods of v1 and 1 pod of v2. The load balancer distributes evenly across all pods — roughly 10% goes to the v2 pod.

Problem: if you want 1% traffic, you need 99 pods of v1 and 1 pod of v2. That is expensive and wasteful.

WITHOUT SERVICE MESH – Canary by Pod Count (Crude)

=====

Want 10% traffic to v2:
Deploy 9 pods of v1 + 1 pod of v2
Kubernetes round-robins: ~10% hits v2

Want 1% traffic to v2?
Deploy 99 pods of v1 + 1 pod of v2 <- too expensive!

Want 0.1% traffic to v2?
Not practical at all.

WITH Service Mesh (Istio Traffic Splitting)

Istio splits traffic by weight, independent of how many pods you run. Run 2 pods of v1 and 2 pods of v2, but send only 10% traffic to v2. Change the weight without touching your deployments at all.

WITH SERVICE MESH – Precise Traffic Split

=====

Running: 3 pods of v1, 3 pods of v2 (same count)
Traffic split set in yaml: v1=90%, v2=10%

Result: 90% of requests go to v1 pods, 10% to v2 pods
No pod count manipulation needed.

Change from 10% to 50%: update one number in yaml
No redeployment. No pod scaling. Instant change.

PROBLEM 4 — Observability: How Do You Know What Is Happening?

WITHOUT Service Mesh

When a user reports "my order failed," the team starts digging through logs. Order-service logs say it sent the request. Payment-service logs say it received the request. But where did the 800ms of latency come from? Which service added the delay? Nobody knows without manually correlating timestamps across log files from 5 different services.

WITHOUT SERVICE MESH – Manual Log Correlation

=====

User reports: order placed but payment took 8 seconds

Engineer checks:

```
order-service log:      "called payment at 14:32:10.100"
payment-service log:    "received request at 14:32:10.250" (150ms gap?)
payment-service log:    "called bank gateway at 14:32:10.280"
bank-gateway log:       ??? (external service, no access to logs)
payment-service log:    "bank responded at 14:32:17.800" (7.5 seconds!)
```

Time spent debugging: 2 hours

Root cause: bank gateway was slow

How to find it: manual log cross-referencing

WITH Service Mesh (Istio Observability)

Istio automatically collects metrics and traces for every request through every service. One dashboard shows the entire request journey with timing.

WITH SERVICE MESH – Instant Visibility

=====

Engineer opens Jaeger trace for the slow order:

Trace: order-123 (total: 8.1s)

```
|-- order-service:          50ms
|  |-- payment-service:      7.8s  <- clearly the bottleneck
|      |-- bank-gateway HTTP call: 7.5s  <- root cause in 10 seconds
```

Engineer opens Kiali (topology):

```
order-service ---> payment-service (P99: 8.1s, red)
|
|      +--> bank-gateway (P99: 7.5s, red)
```

Time spent debugging: 10 seconds

No log reading. No manual correlation. Visual and immediate.

PROBLEM 5 — Access Control: Any Service Can Call Any Service

WITHOUT Service Mesh

Inside the Kubernetes cluster, any service can call any other service. There are no restrictions. Product-service can call payment-service. A random internal tool can call the user database service directly.

This is a security problem. If any service gets compromised, the attacker can reach every other service in the cluster.

WITHOUT SERVICE MESH – No Internal Access Control

=====

product-service ---can call--> payment-service (should NOT be allowed)

analytics-tool ---can call--> user-data-service (should NOT be allowed)

any-compromised-pod -> any service in cluster

Only protection: firewall at cluster edge

Inside the cluster: open network, no restrictions

WITH Service Mesh (Istio Authorization Policy)

Istio lets you define exactly which service can call which other service. Enforced automatically — no code changes required in any service.

WITH SERVICE MESH – Zero Trust Inside Cluster

=====

Policy: only order-service can call payment-service on POST /pay

product-service ---tries to call--> payment-service -> 403 FORBIDDEN

order-service ---calls-----> payment-service -> 200 OK

Enforced at the proxy level before request reaches Spring Boot.

Zero code change in payment-service to enforce this.

THE MENTAL MODEL — What Service Mesh Actually Is

Read this once and you will explain service mesh clearly in any interview.

THE SIDECAR MENTAL MODEL

=====

WITHOUT service mesh – developer owns network logic:

Your Spring Boot Service

+-----+	
Business logic: process payment	
+ Security: TLS config, cert management	<- developer writes this
+ Reliability: retry logic, timeouts	<- developer writes this
+ Observability: metrics, tracing code	<- developer writes this
+ Access control: auth checks	<- developer writes this

```
+-----+
Developer spends 40% of time on network plumbing.
```

WITH service mesh – sidecar owns network logic:

Your Spring Boot Service	Envoy Sidecar (injected by Istio)
+-----+	+-----+
Business logic:	Security: mTLS, certs
process payment	Reliability: retry, timeout
(that is ALL)	Observability: metrics, trace
+-----+	Access control: AuthPolicy
	+-----+

Developer spends 100% of time on business logic.
Sidecar handles all network concerns automatically.
Spring Boot code does not change.

HOW THE SIDECAR GETS THERE:

```
-----
You label the namespace: istio-injection=enabled
You deploy your pod normally.
Istio's admission webhook intercepts the pod creation.
Istio adds an Envoy container to the pod automatically.
Done. No yaml changes in your deployment.
```

SECTION 2 — INTERVIEW CONVERSATIONAL SCRIPT

How to Explain This to an Interviewer

OPENING — When They Ask: "What is a Service Mesh and why do you need it?"

You say:

"Let me answer this with a before-and-after story using Flipkart.

Flipkart has hundreds of microservices — order, payment, inventory, user, notification and more. In the early days, every team was solving the same problems independently.

The payment team added retry logic in Java. The inventory team added retry logic in Go. The notification team added retry logic in Python. All different implementations. When the retry behavior needed to change — say the product team said we need exponential backoff — someone had to update 50 services.

Similarly for security: every service had its own TLS configuration. For observability: every service added its own Prometheus client. For access control: every service had its own authorization checks.

This is the problem service mesh solves. You extract all of that out of every service and into a sidecar proxy — a small Envoy container that runs alongside every pod. The proxy handles retries, mTLS, metrics, tracing, access control — transparently. Spring Boot service makes a plain HTTP call. Envoy intercepts it and handles everything else. Zero code changes.

Istio is the control plane that manages all those Envoy proxies — it injects them into pods, distributes configuration, acts as a certificate authority for mTLS, and collects telemetry."

BEFORE AND AFTER — ONE PICTURE

=====

BEFORE (without service mesh):

50 microservices

Each has: retry code + TLS code + metrics code + auth code
 = $50 \times 4 = 200$ pieces of network logic to write and maintain
 Language-specific: Java, Go, Python, Node.js — all different
 Policy change (e.g. new retry rule): update 50 services

AFTER (with service mesh):

50 microservices

Each has: just business logic
 Network logic lives in Envoy sidecar (same for all languages)
 = 1 yaml file change applies to all 50 services

+--[order-service]--+		+--[payment-service]--+
Spring Boot		Spring Boot
business logic		business logic
+-----+		+-----+
Envoy sidecar	->	Envoy sidecar
retry, mTLS, trace		retry, mTLS, trace
+-----+		+-----+
+--- Istio Control ---+		+-----+
(istiod manages both proxies)		

PART 1 — "How does Spring Boot work with Istio? Do you change your code?"

You say:

"Zero code changes. That is the whole point. Let me show you what actually happens step by step.

Step one: DevOps labels the Kubernetes namespace with istio-injection=enabled. That one label is the only setup needed.

Step two: Your Spring Boot deployment yaml stays exactly the same. One container, your Spring Boot image, port 8080. Nothing about Istio in the yaml.

Step three: When Kubernetes creates the pod, Istio's admission webhook intercepts it. Istio adds an Envoy container to the pod spec automatically. It also injects an init container that sets up iptables rules — these rules redirect all network traffic through Envoy.

Result: your Spring Boot service makes a call to `http://payment-service/pay`. Spring Boot thinks it is going directly to payment-service. Actually, the iptables rule silently redirects that traffic to Envoy on port 15001. Envoy applies all the mesh logic — mTLS, retries, metrics — and forwards to the real payment-service. The Spring Boot code never knows the proxy is there."

WHAT HAPPENS WHEN YOUR SPRING BOOT POD STARTS – STEP BY STEP

=====

YOUR YAML (nothing about Istio):

```
-----
apiVersion: apps/v1
kind: Deployment
spec:
  template:
    service:2.1
    spec:
      containers:
        - name: payment-service
          (injected)
          image: payment-svc:2.1
          ports:
            - containerPort: 8080
# That is it. No sidecar config.
```

WHAT YOUR SPRING BOOT CODE DOES:

```
-----
// OrderService.java
restTemplate.postForObject(
15001)
"http://payment-service/pay",
request, PaymentResult.class);
pod)

Plain HTTP call.
No TLS config.
No retry config.
No tracing code.
of it.
```

WHAT ISTIO CREATES:

```
-----
Pod: payment-service-xxx
|
| Container 1: YOUR CODE
|   image: payment-
|
|   port: 8080
|
| Container 2: ENVOY
|
|   image: istio/proxyv2
|   Intercepts ALL traffic
|
| Init container (runs once):
|   Sets iptables rules:
|   outbound -> Envoy :15001
|   inbound  -> Envoy :15006
```

WHAT ACTUALLY HAPPENS:

```
-----
Order SB -> iptables redirect
-> Envoy (outbound, port

-> mTLS tunnel established
-> Envoy (inbound, payment

-> iptables redirect
-> Payment SB :8080

Encrypted. Retried. Traced.
Zero Spring Boot code for any
```

One-liner:

"Zero code changes. Label the namespace, Istio injects Envoy automatically. iptables redirects all traffic through the proxy — Spring Boot never knows."

PART 2 — "How does mTLS work between services? Where are the certificates?"

You say:

"Let me build this up from basics first.

Normal TLS — like HTTPS in a browser — only the server has a certificate. The browser verifies the server is who it claims to be. But the server does not verify who the browser is. One-way authentication.

mTLS — mutual TLS — both sides have certificates. Server verifies the client. Client verifies the server. So when payment-service receives a request, it verifies: is this really order-service calling me? Not just any pod — specifically the order-service workload.

In Istio, this is fully automatic. istiod runs inside the cluster as a Certificate Authority. When an Envoy proxy starts, it calls istiod and says 'I am the proxy for payment-service, give me a certificate.' istiod verifies this using the Kubernetes service account the pod is running under, then issues a certificate with a SPIFFE identity — a standard format for workload identity, like `spiffe://cluster.local/ns/flipkart/sa/payment-service-sa`.

At runtime, when order-service's Envoy calls payment-service's Envoy, they do a TLS handshake. Order-service presents its certificate. Payment-service presents its certificate. Both verify each other. Then encrypted communication starts. Certificates rotate every 24 hours automatically — no manual renewal, no certbot, no expiry alerts."

mTLS — BEFORE AND AFTER

=====

BEFORE (no mTLS):

order-service -> HTTP -> payment-service
 Anyone on the network can read this traffic.
 Anyone can pretend to be order-service.

AFTER (Istio mTLS):

CERTIFICATE ISSUANCE (happens once at pod startup):

+-----+		+-----+
Envoy		istiod (CA)
(payment pod)		
	CSR	Verifies pod SA
"I run under SA: payment-svc-sa"	----->	Issues cert with
		SPIFFE identity:

```

|                               | <----- | spiffe://cluster |
| Stores cert in               | cert    | .local/ns/flipkart|
| memory (not disk)           |         | /sa/payment-svc-sa|
+-----+                     +-----+
Cert rotates every 24 hours automatically.

```

RUNTIME (every request between services):

```

+-----+ mTLS handshake +-----+
| Envoy                               | Envoy                               |
| (order-service)                    | (payment-service)                  |
| cert: order-svc                    | "Here is my cert"                  |
|                                     | <----- "Here is my cert"         |
|                                     | Both verified                       |
+-----+ Encrypted tunnel +-----+
|                                     |                                     |
| Spring Boot                        | Spring Boot                        |
| (plain HTTP)                      | (plain HTTP)                      |

```

Spring Boot on both sides: zero TLS configuration.

Network between pods: TLS 1.3, encrypted, mutually authenticated.

ENFORCE mTLS WITH ONE YAML (whole namespace):

```

-----
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
  namespace: flipkart
spec:
  mtls:
    mode: STRICT # Reject any non-mTLS traffic in this namespace

```

One-liner:

"istiod is the CA — issues SPIFFE certificates to each Envoy proxy automatically, rotates every 24 hours. Spring Boot code has zero TLS configuration. PeerAuthentication STRICT enforces mTLS across the whole namespace."

PART 3 — "Walk me through Canary Deployment with Istio"

You say:

"Let me show you the before and after because the difference is stark."

Before Istio: Flipkart has payment-service. They build v2.0 with a new payment gateway. They want to test it with 10% of real traffic first. In plain Kubernetes, you do this by running 9 pods of v1 and 1 pod of v2 — the load balancer distributes traffic round-robin, so roughly 10% hits v2. But this is coarse. Want 1%? You need 99 pods of v1. That wastes money and is impractical.

With Istio: you run 3 pods of v1 and 3 pods of v2 — any number you want. Then you write a VirtualService yaml that says route 90% to subset v1 and 10% to subset v2. The weight is

completely independent of pod count. You can set it to 1% without running 99 pods.

Two Istio resources do this. DestinationRule defines what v1 and v2 mean — which pods belong to which subset, identified by a label like version=v1 or version=v2 on the pod. VirtualService sets the traffic percentages. That is it."

CANARY DEPLOYMENT – BEFORE AND AFTER

BEFORE (without Istio):

```
-----
Want 10% traffic to v2:
Run 9 pods v1 + 1 pod v2 = 10 pods total
|
Want 1% traffic to v2:
Run 99 pods v1 + 1 pod v2 = 100 pods total  <- expensive!
|
Want to change from 10% to 20%:
Scale up v2 to 2 pods, scale down v1 to 8 pods = pod manipulation
Takes minutes, blunt, wasteful.
```

AFTER (with Istio):

```
-----
Run any number of pods (e.g. 3 v1 + 3 v2)
Traffic controlled by yaml weight, independent of pod count:
```

STEP 1 – DestinationRule (define what v1 and v2 mean):

```
-----
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: payment-service
spec:
  host: payment-service
  subsets:
    - name: v1
      labels:
        version: v1    # pods with label version=v1
    - name: v2
      labels:
        version: v2    # pods with label version=v2
```

STEP 2 – VirtualService (set traffic percentages):

```
-----
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: payment-service
spec:
  hosts:
    - payment-service
```

```

http:
  - route:
    - destination:
        host: payment-service
        subset: v1
        weight: 90      # 90% of traffic
    - destination:
        host: payment-service
        subset: v2
        weight: 10      # 10% of traffic

```

ROLLOUT (just change the weight number – nothing else):

```

-----
Day 1:  v1=90, v2=10   (watch error rates on Grafana)
Day 3:  v1=50, v2=50   (looks good, increase exposure)
Day 5:  v1=0,  v2=100  (full rollout)
Day 6:  delete v1 deployment

```

Traffic flow:

order-service Envoy reads VirtualService -> routes 90% to v1 pods, 10% to v2 pods

No change to order-service, no change to payment-service code.

One-liner:

"VirtualService weights control the traffic split independently of pod count. DestinationRule maps subsets to pod labels. Change weight, no redeployment needed."

PART 4 — "How do retries, timeouts, and circuit breaker work?"

You say:

"Let me walk through each one. All three are configured in Istio yaml files — none require any Spring Boot code change.

Retries: I configure them in VirtualService. For calls to payment-service, retry up to 3 times if you get a connection failure or 503, with a 2-second limit per attempt. Envoy handles this — order-service calls payment-service once in its Java code. Envoy does up to 3 attempts silently. Order-service receives either a success or a final failure.

Timeout: also in VirtualService. If payment-service does not respond within 5 seconds total, Envoy returns a 504 to the caller. No need to set timeouts in every service that calls payment.

Circuit breaker: this one is important. Imagine payment-service has 4 pods. One pod has a bad deployment — it throws a 503 on every request. Without a circuit breaker, every third or fourth request to payment-service hits that bad pod and fails. With Istio's outlier detection in DestinationRule, after 5 consecutive failures from that pod, Envoy ejects it from the load balancing pool for 30 seconds. The other 3 pods keep serving normally. After 30 seconds, Istio probes the bad pod. If it is healthy, it comes back. If not, the ejection time doubles.

Key point: the circuit breaker is at the individual pod level, not the whole service. 3 out of 4 pods keep working even while one is ejected."

RETRIES + TIMEOUT + CIRCUIT BREAKER – BEFORE AND AFTER

=====

BEFORE (without Istio):

```
// OrderService.java – developer writes all of this
public PaymentResult callPayment(PaymentRequest req) {
    int attempts = 0;
    while (attempts < 3) {
        try {
            return paymentClient.call(req);
        } catch (Exception e) {
            attempts++;
            if (attempts == 3) throw new PaymentException(e);
            Thread.sleep(100L * attempts);
        }
    }
    // timeout? set on RestTemplate
    // circuit breaker? add Resilience4j dependency
    // configure bean, add @CircuitBreaker annotation...
}
// Write this for every service. Maintain it. Keep consistent across
languages.
```

AFTER (with Istio – all in yaml, zero Java code):

```
VirtualService (retries + timeout):
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
spec:
  hosts: [payment-service]
  http:
    - timeout: 5s                # total request timeout
      retries:
        attempts: 3              # retry up to 3 times
        perTryTimeout: 2s        # each attempt max 2s
        retryOn: "connect-failure,503"
      route:
        - destination:
            host: payment-service
```

```
DestinationRule (circuit breaker / outlier detection):
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
spec:
  host: payment-service
  trafficPolicy:
```

```

outlierDetection:
  consecutive5xxErrors: 5    # eject pod after 5 consecutive errors
  interval: 10s              # check every 10 seconds
  baseEjectionTime: 30s      # eject for minimum 30 seconds

```

HOW CIRCUIT BREAKER WORKS (pod-level ejection):

```

payment-service: 4 pods
pod-A: healthy   pod-B: healthy   pod-C: BAD   pod-D: healthy

```

Requests to pod-C: 503, 503, 503, 503, 503 (5 consecutive errors)

|

Istio ejects pod-C from load balancing pool

|

Traffic now: only pod-A (33%), pod-B (33%), pod-D (33%)

pod-C gets ZERO traffic

|

After 30 seconds: Istio sends ONE probe request to pod-C

If 200 OK: pod-C re-enters pool

If 503: ejection time doubles (60s, 120s...)

```

Order-service code: return paymentClient.pay(request);  <- still just one
line

```

One-liner:

"Retries and timeout in VirtualService, circuit breaker via outlier detection in DestinationRule. All yaml. Pod-level ejection — healthy pods keep serving while bad pods are isolated."

PART 5 — "How does Istio give you observability?"

You say:

"Three tools and three layers. Let me explain the before and after first."

Before Istio: an engineer gets a report that orders are slow. They open four different log files from four different services. They manually correlate timestamps. It takes an hour to find that the bank gateway is responding slowly.

With Istio: every Envoy proxy automatically emits metrics for every request it handles. Prometheus scrapes these metrics from every proxy. Grafana shows me dashboards — error rate per service, P99 latency per service-to-service edge, requests per second. No Spring Boot Micrometer code needed for these network metrics.

For distributed tracing: Envoy generates a trace ID for every incoming request and propagates it as an HTTP header — X-B3-TraceId. As the request passes through order-service to payment-service to bank-gateway, Envoy on each hop adds its span to the trace. Jaeger collects all these spans and shows the full journey with timing. There is one thing Spring Boot must do: when it receives a

request with trace headers, it must forward those same headers in any outbound calls it makes. Spring Cloud Sleuth or Micrometer Tracing does this automatically.

For topology: Kiali reads the Prometheus data and draws a live map of all your services with error rates and latency on every edge. During a Big Billion Days incident, Kiali shows you in 10 seconds exactly which service-to-service call is throwing errors."

OBSERVABILITY – BEFORE AND AFTER

=====

BEFORE (without Istio):

// Every Spring Boot service needs this code:

```
@Bean
MeterRegistry meterRegistry() { return new PrometheusMeterRegistry(...);
}
```

```
@Timed("payment.call.duration")
public PaymentResult pay(...) { ... }
```

```
// Plus tracing setup in every service:
// Add spring-cloud-sleuth dependency
// Configure zipkin endpoint
// Manually create spans for important operations
```

// 50 services = 50 times this setup. Different versions. Different configs.

AFTER (with Istio – automatic for all services):

LAYER 1: METRICS (Envoy generates automatically)

```
+-----+
| Envoy exposes at :15090/stats/prometheus |
| istio_requests_total{                     |
|   source_workload="order-service",        |
|   destination_workload="payment-service", |
|   response_code="200"                     |
| }                                          |
| istio_request_duration_milliseconds_bucket{... } |
+-----+
```

Prometheus scrapes this from EVERY pod automatically.

Grafana shows:

– payment-service: 2,847 req/s, 0.02% errors, P99: 145ms
Zero Spring Boot code for these metrics.

LAYER 2: TRACES (Envoy propagates, Jaeger collects)

+-----+

User request -> order-service -> payment-service -> bank

```

Envoy adds header: X-B3-TraceId: abc123
Envoy adds span:    order-service 50ms

Spring Boot ORDER-SERVICE must forward headers:
// With Micrometer Tracing (auto-configured in Spring Boot 3):
// Header forwarding is automatic when using WebClient or RestTemplate
// Just add micrometer-tracing dependency

Envoy adds span:    payment-service 7.8s
Envoy adds span:    bank-gateway 7.5s

Jaeger shows:
Trace abc123 (8.1s total):
|-- order-service:      50ms
|   |-- payment-service: 7800ms
|       |-- bank-gateway:7500ms  <- you see this instantly

LAYER 3: TOPOLOGY (Kiali)
+-----+
[order-service]
  | 145ms, 0.02% error
  v
[payment-service]
  | 7500ms, 0.5% error  <- RED EDGE
  v
[bank-gateway]
Visual. Real-time. No log reading needed.

```

One-liner:

"Envoy generates network metrics automatically — Prometheus scrapes, Grafana displays. Spring Boot needs to forward B3 trace headers (Micrometer Tracing does this). Kiali shows live topology with error rates on every service edge."

PART 6 — "How do you control which services can call each other?"**You say:**

"This is access control at the network level. Let me show you why it matters.

Without Istio, any pod in the cluster can call any other pod. If product-service gets compromised through a vulnerability, the attacker running inside that pod can call payment-service directly. Nothing blocks them.

With Istio, each Envoy has a SPIFFE identity — a certificate that says which service it is. AuthorizationPolicy lets you say: payment-service will ONLY accept calls from order-service and refund-service. Anyone else — product-service, analytics service, a compromised pod — gets 403 at the Envoy layer. The request never reaches the Spring Boot container.

The recommended pattern is deny-all by default. You create one AuthorizationPolicy with an empty spec — that blocks all traffic in the namespace. Then you create explicit ALLOW policies for each legitimate service pair. This way, if you deploy a new service and forget to write its policy, it is blocked by default. You have to explicitly open connections, not explicitly close them."

AUTHORIZATION POLICY – BEFORE AND AFTER

=====

BEFORE (without Istio):

Any pod -> any other pod. No restrictions.
Compromised product-service pod can call payment-service directly.
Attacker has access to all services.

AFTER (with Istio AuthorizationPolicy):

STEP 1: Deny all traffic by default (namespace-wide)

```
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: deny-all
  namespace: flipkart
spec: {} # empty = deny ALL incoming traffic in this namespace
```

STEP 2: Explicitly allow order-service to call payment-service

```
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: allow-order-to-payment
  namespace: flipkart
spec:
  selector:
    matchLabels:
      app: payment-service
  action: ALLOW
  rules:
    - from:
        - source:
            principals:
              - "cluster.local/ns/flipkart/sa/order-service-sa"
              - "cluster.local/ns/flipkart/sa/refund-service-sa"
      to:
        - operation:
            methods: ["POST"]
            paths: ["/api/v*/pay"]
```

RESULT:

```

-----
order-service    -> POST /api/pay    -> payment-service 200 OK (allowed)
product-service  -> POST /api/pay    -> payment-service 403   (blocked)
compromised-pod  -> POST /api/pay    -> payment-service 403   (blocked)

```

Blocked at Envoy. Spring Boot code in payment-service never executes.
Zero code change in payment-service.

One-liner:

"deny-all policy by default, then explicit ALLOW per service pair on specific paths. Enforced at Envoy using SPIFFE certificate identity. Zero code in any service."

FULL PICTURE — Flipkart With and Without Service Mesh

FLIPKART ARCHITECTURE — COMPLETE BEFORE AND AFTER

WITHOUT SERVICE MESH:

order-service (Java)

```

+-----+
| processOrder() {
|   // retry logic (Java)
|   // TLS client config
|   // metrics counter increment
|   // trace span creation
|   // check authorization header
|   paymentService.pay(...)
|   inventoryService.reserve(...)
| }
+-----+

```

Same story for 49 other services.
Different languages. Different implementations.
Policy change = update 50 services.

WITH SERVICE MESH:

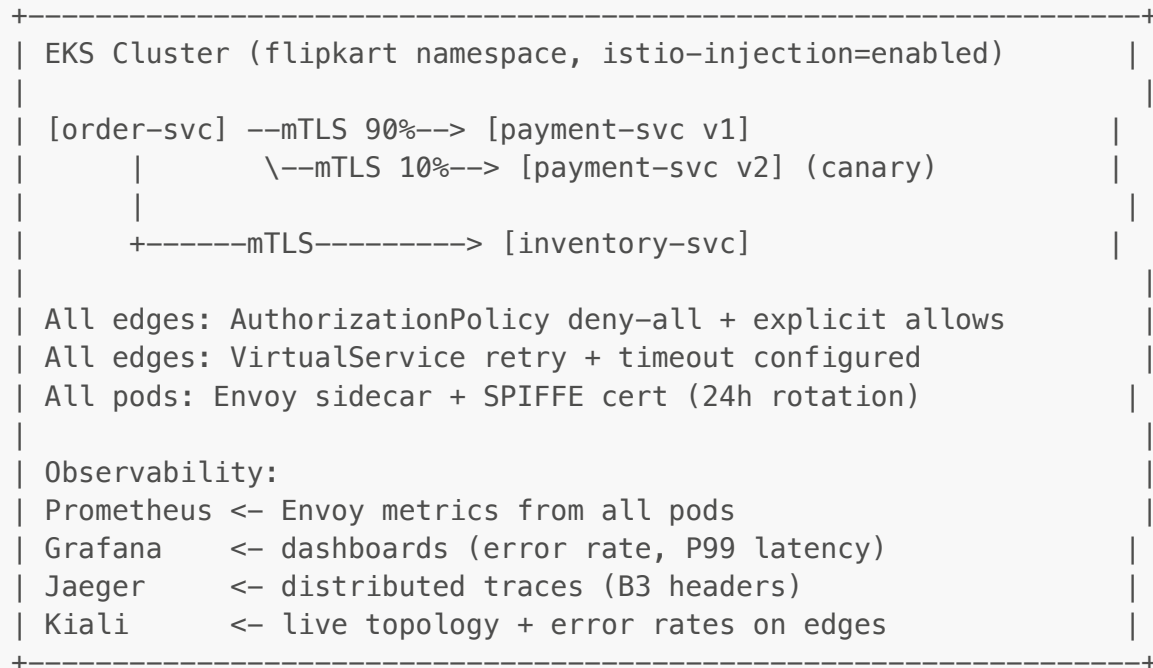
order-service (Java)

```

+-----+
| processOrder() {
|   paymentService.pay(...) | <- just this
|   inventoryService.reserve() | <- and this
| }
+-----+

```

Envoy proxy (injected automatically)



charged twice.

Two-part fix. First: configure `retryOn` carefully — only retry on connect-failure and reset, which mean the request never reached the server. Never retry on 503 for write operations, because 503 means the server received the request and may have processed it before failing.

Second: make the payment service idempotent. The caller sends an `Idempotency-Key` header with a UUID. The payment service stores the result in Redis keyed by that UUID. If the same key comes in again — due to a retry — it returns the cached result without charging again."

DUPLICATE PAYMENT – THE PROBLEM AND FIX

=====

DANGEROUS CONFIG (retries on 503 for writes):

order -> POST /pay -> payment processes charge -> crashes -> 503

Envoy retries:

order -> POST /pay -> payment processes charge AGAIN -> 200

Customer charged TWICE. Both callers got 200. Silent data corruption.

SAFE RETRY CONFIG for payment calls:

retries:

attempts: 2

retryOn: "connect-failure,reset" # safe: request never reached server

NOT "503" for write operations

IDEMPOTENCY KEY in Spring Boot:

// Caller (order-service) generates once per order attempt:

```
String idempotencyKey = UUID.randomUUID().toString();
```

```
headers.set("Idempotency-Key", idempotencyKey);
```

// Payment service checks before processing:

```
@PostMapping("/api/pay")
```

```
public ResponseEntity<PaymentResult> pay(
```

```
    @RequestHeader("Idempotency-Key") String idempKey,
```

```
    @RequestBody PaymentRequest req) {
```

```
    String cached = redis.get("pay:idem:" + idempKey);
```

```
    if (cached != null) {
```

```
        return ResponseEntity.ok(deserialize(cached)); // no charge
```

```
    }
```

```
    PaymentResult result = gateway.charge(req); // charge once
```

```
    redis.set("pay:idem:" + idempKey, serialize(result),
```

```
    Duration.ofHours(24));
```

```
    return ResponseEntity.ok(result);
```

```
}
```

Even if retried 5 times: customer charged ONCE.

First call stores result in Redis.
All retries return cached result.

TRAP-2 — "Do you need Resilience4j if you have Istio?"

You say:

"They work at different layers and you want both.

Istio works at the network layer — it does not know your business logic. It can retry a request, but it cannot fall back to a cached product price if the pricing service is down. It can eject a bad pod, but it cannot decide to return a default recommendation when the ML service is slow.

Resilience4j works inside the Java process — it has access to your application context. You can write fallback logic that uses a cache, a default value, or a degraded response. That is application-level resilience.

The split is: Istio handles infrastructure concerns — pod ejection, network retries, timeout, cross-language circuit breaking. Resilience4j handles business concerns — fallback logic, application-context decisions. Use both together."

ISTIO vs RESILIENCE4J — DIFFERENT LAYERS

=====

Network layer (Istio):

```
+-----+
| payment-service has 4 pods          |
| pod-C is returning 503 on all requests |
| Istio ejects pod-C after 5 errors    |
| Requests route to pod-A, pod-B, pod-D |
+-----+
```

Resilience4j cannot do this — it does not see individual pods.

Application layer (Resilience4j):

```
+-----+
| ALL pods of pricing-service are slow |
| Resilience4j opens circuit in caller |
| Falls back: return cached price from  |
|   Redis instead of calling pricing    |
+-----+
```

Istio cannot do this — it does not know about Redis or cached prices.

USE BOTH:

Istio: pod-level ejection, network retry, cross-language
Resilience4j: business fallback, application-context logic

TRAP-3 — "When would you NOT use a service mesh?"

You say:

"If you have fewer than 5 or 6 services, a service mesh adds more overhead than value. You are adding an Envoy sidecar to every pod, running istiod as a control plane, learning a new set of CRDs. For 3 services, it is genuinely easier to configure TLS and retries directly in each service.

The inflection point is roughly 10+ services across multiple teams, or when security requirements demand mTLS and you do not want to manage certificates manually in every service. For multicloud or hybrid deployments where services run in different clusters or clouds, service mesh shines because it gives you a consistent network policy layer across all environments."

PART 7 — "Walk me through istiod — what does the control plane actually do?"

You say:

"istiod is the single control plane component in Istio since version 1.5. Before 1.5, Istio had four separate components — Citadel handled certificates, Mixer handled telemetry and policy, Galley handled configuration validation, Pilot handled proxy configuration. In 1.5 they merged everything into one istiod binary to simplify operations.

istiod does four things. First, it is a Certificate Authority — it issues and rotates SPIFFE certificates for every Envoy proxy. Second, it is a service registry — it watches the Kubernetes API and maintains an up-to-date list of every service and endpoint in the cluster. Envoy proxies query istiod for this to know where to send traffic. Third, it is a configuration distributor — it reads the VirtualService, DestinationRule, AuthorizationPolicy CRDs you create and converts them into Envoy-specific xDS configuration, then pushes that config to all proxies. The proxies do not need to query istiod at request time — they have the full config locally and talk to each other independently. Fourth, it collects telemetry from Envoy proxies and exposes it for Prometheus to scrape."

istiod — FOUR RESPONSIBILITIES

=====

istiod

+-----+-----+-----+		
1. Certificate Authority (CA)		
Issues SPIFFE certs to each Envoy on startup		
Rotates every 24 hours automatically		
2. Service Registry		
Watches Kubernetes API server		
Tracks: which services exist, which pods are healthy		
Envoy queries this to route traffic		
New service deployed? Auto-registered. No config.		
3. Config Distributor (xDS protocol)		
Reads: VirtualService, DestinationRule, AuthPolicy		
Converts to Envoy-native config		


```

| Pushes to all proxies in the cluster |
| Proxies work independently after receiving config |
|
| 4. Telemetry Collection |
| Envoy proxies report metrics/traces to istiod |
| Prometheus scrapes from istiod (or directly from pod) |
+-----+

```

BEFORE v1.5 (multiple components):	SINCE v1.5 (one component):
-----	-----
Citadel (certificates)	istiod (all of the above)
Mixer (telemetry + policy)	
Galley (config validation)	
Pilot (proxy config push)	
= 4 pods to run, 4 configs to manage	= 1 pod, simple to operate

CONTROL PLANE vs DATA PLANE:

Control plane (istiod): manages configuration, certs, registry
 Data plane (all Envoy sidecars): actually handle request traffic

istiod does NOT sit in the request path.
 When order-service calls payment-service — istiod is NOT involved.
 Envoy-to-Envoy communication is direct. istiod only pushes config.

One-liner:

"istiod is one binary doing four jobs: CA for certs, service registry, config push to Envoy proxies via xDS, and telemetry collection. It is NOT in the request path — proxies handle traffic independently using config they already have."

PART 8 — "How does service discovery work? Do you configure endpoints manually?"

You say:

"Without Istio, you often had to configure endpoints explicitly — like storing the URL of payment-service in order-service's config map. If payment-service moved to a different port or IP, you updated config everywhere.

With Istio, service discovery is automatic. istiod watches the Kubernetes API server. The moment you deploy payment-service and its pods come up healthy, istiod registers it in its internal service registry. Envoy proxies query this registry and instantly know where to send traffic for payment-service.

In practice, Spring Boot just calls `http://payment-service/pay` — that is the Kubernetes DNS name. istiod ensures Envoy knows the current healthy endpoints behind that name. If one pod crashes and a new one comes up, istiod updates the registry and all proxies know within seconds. Zero manual configuration."

SERVICE DISCOVERY – BEFORE AND AFTER

=====

BEFORE (manual endpoint config):

order-service application.yaml

payment:

url: http://10.0.1.45:8080 # hardcoded IP of payment pod

If payment pod restarts with new IP: update config, redeploy

Or with config map:

payment:

url: http://payment-service:8080 # kubernetes DNS (better)

Still need to manage this for every service pair

AFTER (Istio automatic service discovery):

istiod watches Kubernetes API:

kubectl apply -f payment-deployment.yaml

|

Kubernetes creates pods with label app=payment-service

|

istiod detects new endpoints via K8s watch

|

istiod updates service registry:

payment-service -> [pod-A:8080, pod-B:8080, pod-C:8080]

|

istiod pushes update to all Envoy proxies (xDS)

|

order-service Envoy now knows: payment-service = 3 endpoints

order-service Spring Boot code:

restTemplate.postForObject("http://payment-service/pay", ...)

Kubernetes DNS resolves payment-service

Envoy intercepts, checks service registry, routes to healthy pod

Pod crashes? istiod removes it from registry within seconds.

New pod up? istiod adds it. All automatic.

One-liner:

"istiod watches the Kubernetes API and auto-registers all services. Envoy gets current endpoints via xDS push. Spring Boot just uses the Kubernetes DNS name — zero manual endpoint configuration."

PART 9 — "What is Istio Ingress Gateway and how is it different from nginx ingress?"

You say:

"Istio Ingress Gateway is the entry point for traffic coming into your Kubernetes cluster from outside. Think of it as Istio's replacement for the nginx ingress controller — it is an Envoy proxy running as a dedicated pod that accepts external traffic on ports 80 and 443.

The key difference is how you configure it. Nginx ingress uses Ingress objects. Istio uses two CRDs: Gateway and VirtualService. The Gateway CRD configures what the gateway pod listens on — ports, protocols, TLS termination. The VirtualService then says what to do with traffic that arrived at the gateway — route to which service, rewrite paths, apply headers.

On AWS EKS, Istio Gateway creates an AWS Network Load Balancer automatically using the AWS Load Balancer Controller. External traffic hits the NLB, NLB forwards to the Gateway pod, Gateway evaluates VirtualService rules and routes to your Spring Boot service inside the cluster. At that point it enters the mesh and gets all the mTLS, retry, and observability features."

ISTIO INGRESS GATEWAY – HOW IT WORKS ON EKS

=====

OUTSIDE CLUSTER

User browser

|

| HTTPS

v

AWS Network Load Balancer

(auto-created by AWS LB Controller)

|

| TCP passthrough

v

Istio Gateway Pod (Envoy)

Configured by Gateway CRD:

– port: 443

– protocol: HTTPS

– TLS: terminate here

|

| Evaluates VirtualService rules

v

order-service pod

(Envoy sidecar)

mTLS from here onward

payment-service pod

(Envoy sidecar)

mTLS from here onward

GATEWAY CRD (what port/protocol to listen on):

apiVersion: networking.istio.io/v1beta1

kind: Gateway

metadata:

 name: flipkart-gateway

spec:

 selector:

```

istio: ingressgateway    # select the gateway pod
servers:
  - port:
      number: 443
      name: https
      protocol: HTTPS
    tls:
      mode: SIMPLE
      credentialName: flipkart-tls-cert    # K8s secret
    hosts:
      - "flipkart.com"

```

VIRTUALSERVICE (where to route after gateway):

```

-----
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: flipkart-routes
spec:
  hosts:
    - "flipkart.com"
  gateways:
    - flipkart-gateway
  http:
    - match:
        - uri:
            prefix: /api/orders
      route:
        - destination:
            host: order-service
            port:
              number: 8080
    - match:
        - uri:
            prefix: /api/payment
      route:
        - destination:
            host: payment-service
            port:
              number: 8080

```

NGINX INGRESS vs ISTIO GATEWAY:

```

-----
Nginx Ingress:    L7 only, limited traffic rules, no mesh integration
Istio Gateway:    L4+L7, full VirtualService rules, integrated with mesh
                  mTLS, retries, canary all work from the gateway inward

```

One-liner:

"Gateway CRD configures the entry point (ports, TLS). VirtualService routes from gateway to services. On EKS, creates an AWS NLB automatically. Traffic enters mesh from there — all mTLS

and retry features apply."

QUICK REFERENCE — Key Facts for Interview

Topic	Plain English	Technical Detail
What is service mesh	Sidecar proxy handles network concerns for all services	Envoy proxy injected by Istio sidecar admission webhook
Spring Boot changes	Zero — not a single line of code	Namespace label + iptables redirect in init container
mTLS	Both services verify each other's identity + encrypted traffic	SPIFFE certs issued by istiod CA, rotated every 24h
Canary deployment	Split traffic by percentage, not by pod count	VirtualService weights + DestinationRule subsets
Retries	Proxy retries automatically, your code calls once	VirtualService retryOn field, perTryTimeout
Circuit breaker	Ejects individual bad pods, not whole service	DestinationRule outlierDetection consecutive5xxErrors
Observability	Auto metrics from proxy, traces from headers	Envoy -> Prometheus -> Grafana; B3 headers -> Jaeger; Kiali topology
Access control	Define which service can call which	AuthorizationPolicy deny-all + explicit ALLOW with SPIFFE principals
Duplicate payment trap	Retry + non-idempotent write = double charge	retryOn: connect-failure only + Idempotency-Key in Redis
Resilience4j vs Istio	Both needed, different layers	Istio: pod ejection, Resilience4j: business fallback